

Linux Fundamentals

Introduction:

Linux is a powerful Unix-based Operating System used by millions of people around the world. One of the most important aspects of Linux is its file system.

The file system is a way of organizing and storing the files on a computer.

Linux is used in cloud computing to configure the web servers (as there is no GUI to interact with these servers), modern software development etc.

Linux helps in automation of workflows as automating the textual commands is easier than clicking through the graphical interfaces. It is free and open-source, less resource consuming and modern development tools like Docker, Kubernetes and CI/CD pipelines are built natively around Linux environments.

That's why Linux is very useful and we should learn the commands which are used to interact with a Linux system.

Absolute path : It is the path to a directory or file which is given from the root to that directory or file. So, it is the complete path of a file or folder from the root.

Relative path: It is a path which is not given from the root, it's given from the current directory. So, this path is relative to the current directory.

The commonly used Linux commands are as follows.

1. **pwd** : this is print working directory command, it will print the current working directory, where we are currently present.
2. **ls** : this is the list command which lists all the available files and folders/directories in the current directory.
 - **ls -a** : This command prints all the files and folders including the hidden ones(whose names starts with a dot '.').
 - **ls -l** : It lists all the files in a long format, including the file permissions, owner, group, and last modified date.
 - **ls -lh** : this will list the files along with their size .
 - **ls -r** : lists the files in a reverse order.
 - **ls -t** : lists file sorted based on time with most recently modified file first.
 - **ls -u** : lists files sorted based on user with the files owned by the current user first.
 - **ls -R** : this will print all the files and folders which are present in deeper directories giving us an overview of the complete hierarchy of files.
3. **mkdir** : The make directory (mkdir) command is used to create a new directory or folder.
Example : mkdir dir1

We can also use **mkdir -v dir1** , here the -v(verbose) flag gives a confirmation message after creating the directory while the normal mkdir doesn't give it.

- **mkdir dir1 dir2 dir3** or **mkdir -v dir1 dir2 dir3**: In this manner, we can create multiple folders in one command. These folders are present at the same level in the current directory.
- **mkdir -p dir1/dir2/dir3** or **mkdir -vp dir1/dir2/dir3**: In this command the flag -p will create the parent directories and we can create nested directories in one go.

4. **cd** : The change directory (cd) command is used to change the directory.

- **cd <folder name>** : the cd followed by a folder name in the current directory will move us inside the specified directory (enter a folder via specifying its name).
- **cd ..** : This will take us out of the current directory to one folder back (come out of the folder).
- **cd -** : This will take us forward to the previous working directory (move back to the same folder from where we have just come out).
- **cd ../..** : This will take us two directories back from the current directory (move two folders back from the current folder).
- **cd /** : This will take us to the root directory of the complete file system.
- **cd** : the cd command without any arguments will take us to the home directory of the user.

5. **cp** : This is Copy (cp) command which is used to copy a file or folder from one directory to another and it can also be used to copy the contents of a file to another.

To copy the contents of a file into another the syntax is

cp <path of file 1> <path of file 2> for example,

cp file1.txt file2.txt considering both files are in the same folder, otherwise we have to specify the paths of the files.

Result: Contents of file1 are copied to the file2.

To create a copy of a file from one directory and pasting it to another directory, the syntax is **cp <file_path> <directory_path>**

cp file1.txt path/to_the/destination_folder

Result: The file1 will be copied to the destination folder.

If there is a file name at the end of path then the contents of the file will be copied and if there is a folder in the end of path, that folder is going to be copied.

- **cp -a** : This command will copy all the files and folders recursively from the directory including the hidden files (names starting from a dot). It preserves the original file permissions, timestamps, user ownership etc.
cp -a source_folder_path destination_folder_path.
- **cp -r** : This command will also copy all the files and folders recursively from the directory including the hidden files (names starting from a dot). It does not preserve the original file

permissions, timestamps, user ownership etc. The copied files get new metadata.

cp -r source_folder_path destination_folder_path.

- **cp -v** : this is the verbose mode in which a successful message is displayed for all the copied files. It can be combined with -a and -r flags.

6. **mv** : This is the move command used to move the file content from one file to another or move the complete folder from one directory to another. **mv <path_of_file1> <path_of_file2>**

Example: mv file1 /path/to/destination_file/file2.txt

Result: Contents of file1 will be cut and pasted in file2 in destination folder.

mv file1 /path/to/destination_folder

Result: The file1 will move to the destination folder.

mv folder1 /path/to/destination_folder

Result: folder1 will move to the destination folder.

It can also be used to rename a file or folder, if we move the file or folder and paste it in the same directory with a different name.

Example: mv file1.txt newFile.txt

Result : file1 contents were moved and pasted in a file named newFile.txt in the same folder.

mv old_folder_name new_folder_name

Result: the old folder will cut and pasted in the same destination with a new name.

or

mv folder1 path/to/destination_folder/folder2

Result: The folder1 will be moved to destination and will be renamed to folder2.

7. touch : This command is used to create a new file. For example :

touch file1.txt

touch readme.md

8. echo : This command is used to print some text on the screen and it is also used to add or append text in a file. For example:

echo "hello world" output -> hello world

echo "hello world" > file1.txt -> this will add the hello world text in the file1.txt.

echo "I'm learning Linux" >> file1.txt -> this will append the text to this file.

9. cat : this command is used to run a file. For example:

cat file1.txt

output : hello world I'm learning Linux.

10. head : this command prints an upper portion of the file.

Example :

head -n 2 file.txt -> this will print the upper 2 lines of the file only.

11. tail : this will print the lower portion of the file. Example:

tail -n 5 file.txt -> this will print the lower 5 lines of the file only.

12. rm : This is remove command which is used to delete a file.

for example: **rm file1.txt**

13. rmdir : This is remove directory command which can be used to delete an **empty** directory. For example: **rmdir dir2**

- **rmdir -r** : this command recursively deletes the content of a directory so first it deletes the files and then the folder containing it. So it allow us to delete a non-empty folder.
- **rmdir -rf** : this command forcefully deletes a folder and its contents.
- **rmdir -rf /** : this command is dangerous and should not be used as it will delete all the files from the root directory which will lead to system corruption.

14. clear : This command is used to clear the screen. It does not deletes the history but moves the terminal cursor to the top of the screen.

15. tar : this stands for tape archive command in Linux which is used to create archive files by zipping up and compressing a bunch of files.

- **tar -cf <archive_name> file1.txt file2.txt**

This command will zip the files mentioned into an archive file with the name specified.

- **tar -zcf <archive_name> file1.txt file2.txt**

This command will compress and zip the files into an archive file, this reduces the size of the archived file as compared to the above command.

- **tar -xvzf <archive_name>**

This command is used to view the files present in the archive file.

- **tar xf <archive_name> -C <output_folder>**

This command extracts the archive into a folder. The folder should be present before extraction.

Piping: In Linux, piping is done with the help of pipe symbol '|'. It involves the passing of output of one command as an input to the next command. **Example:**

ls | grep pictures

In this example, the output of ls command which is to list all the directories and files will be given to the next command via piping (|). The second command called **grep** is used to search a sub-string. Here we have searched word 'pictures'.

Now it will list us all the files and folders which includes 'pictures' string in their name.

Dumping: It is the process of adding some generated information in a file. We achieve this using > and >> symbols.

ls > dump.txt -> This command will dump all the listed directories and files in the file instead of printing it on screen. We can store logs or other content in a file in the same way.

It deletes the previous data in the file.

If we don't want to delete the previous data, so instead of dumping the data we can append the data to the file using >>

Example:

pwd > dump.txt -> adds the current working directory information in the file.

ls >> dump.txt -> adds the list of files and folder in the file without removing the previous information.

Boolean logics with commands: Using Boolean logics we can combine multiple commands to execute in one go. Example: Executing a c++ file involves two steps, first compiling the file and second is to run the executable file.

Using **&&** symbol we can combine the commands to compile a c++ file and execute the file.

<command 1> && <command 2>

g++ file.cpp --std=c++14 -o temp **&&** ./temp

Vim : vim is a free open source text-editor present on the terminal.

There are three modes in which vim operates.

- i) Normal mode: This is the default start-up mode of vim used for navigating, deleting, copying files. We can only read the files and we can't edit the file in this mode.
- ii) Insert mode: In this mode we can edit the file. We can enter this mode by pressing 'i'. To exit this mode we can use 'esc' button.
- iii) Command-Line (Last line) mode: It is entered by pressing ':' from the normal mode to perform operations like saving or quitting.

To exit vim,

- i) From normal mode : we can press :q (colon + q).
- ii) From Insert mode : to save the changes and exit we can use :wq or :x
To exit without saving the changes we can use :q!

To create a file,

Use vim **new_file.txt** or **vi new_file.txt** command. It will open the file if it's already present in the directory or it will create the file with this name if it's not present.

In normal mode,

- **to navigate in vim**, we can use keys h(move left), j (move down), k (move up) and L (move right). Alternatively, we can also use up, down, left and right keys.
- Upon reaching to a line in a file, we can **delete a complete line** in one go using '**dd**'.
- To undo the last action we can use '**u**'.
- To redo the changes we can use **ctrl + r**.

- We can use **'yy'** or **'shift+Y'** to copy a complete line.
- Use **'yw'** to copy a single word.
- Use **'p'** to paste the copied content.
- Navigational shortcuts
 - i) **gg** : move cursor to the starting of the first line in the file.
 - ii) **G (Capital)**: move cursor to the starting of the last line in the file.
 - iii) **w** : it will move the cursor one word forward.
 - iv) **2w** : it will move the cursor two words forward and so on(3w, 4w etc.)
 - v) **d2w** : it will delete two words together and so on.
- To find and replace a specific word we can use **:%s/PREVIOUS_WORD/NEW_WORD**